

Function Block Diagram (FBD)

Logic Blocks | Signal Flow | Timers | Counters | vs Ladder

What You Will Learn

- FBD signal-flow concept: inputs → logic blocks → outputs, left to right
- Basic logic blocks: AND, OR, NOT, XOR and how to connect them
- Using TON timer and CTU counter blocks in FBD programs
- When to choose FBD over Ladder and how to combine both in one project

FBD Block Anatomy

IN1 ■■

IN2 ■■ AND ■■■■ OUT

IN3 ■■

Awet G. Nway

Founder, AYE Tech Hub | PLC & Industrial Automation Engineer

ayetechub.com | © 2026 AYE Tech Hub | Free Educational Resource

PLC-008: Function Block Diagram (FBD)

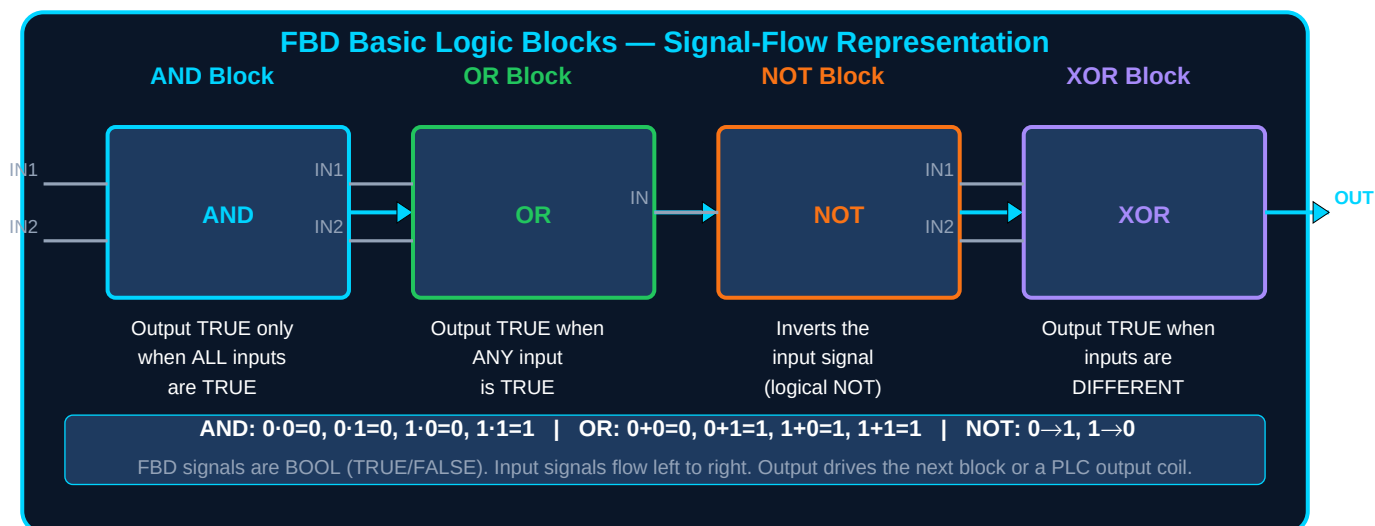
Function Block Diagram is the second of the five IEC 61131-3 languages, and it is the most natural language for engineers coming from an analogue electronics or signal processing background. While Ladder Diagram represents logic as relay contacts on rungs, FBD represents it as connected signal-flow blocks — similar to how a signal processing engineer would draw a block diagram on a whiteboard. Data flows from left to right through blocks that process it.

Learning Objectives

- Explain the FBD signal-flow concept and draw a simple FBD network from a verbal description
- Use the four basic logic blocks (AND, OR, NOT, XOR) and connect them correctly
- Place and configure TON timer and CTU counter blocks in an FBD program
- Design a motor start/stop FBD program using OR, AND, NOT, and SR blocks
- Identify when FBD is preferable to Ladder Diagram for a given application

1. FBD Fundamentals — Signal-Flow Programming

In FBD, every piece of logic is represented as a rectangular block with inputs on the left side and outputs on the right side. Signals flow from left to right through wires connecting blocks. A block performs a specific function on its inputs and produces an output that can drive another block or a PLC output variable. The visual appearance is similar to electronic circuit block diagrams — intentionally so, to make the language accessible to engineers who think in terms of signal flow.



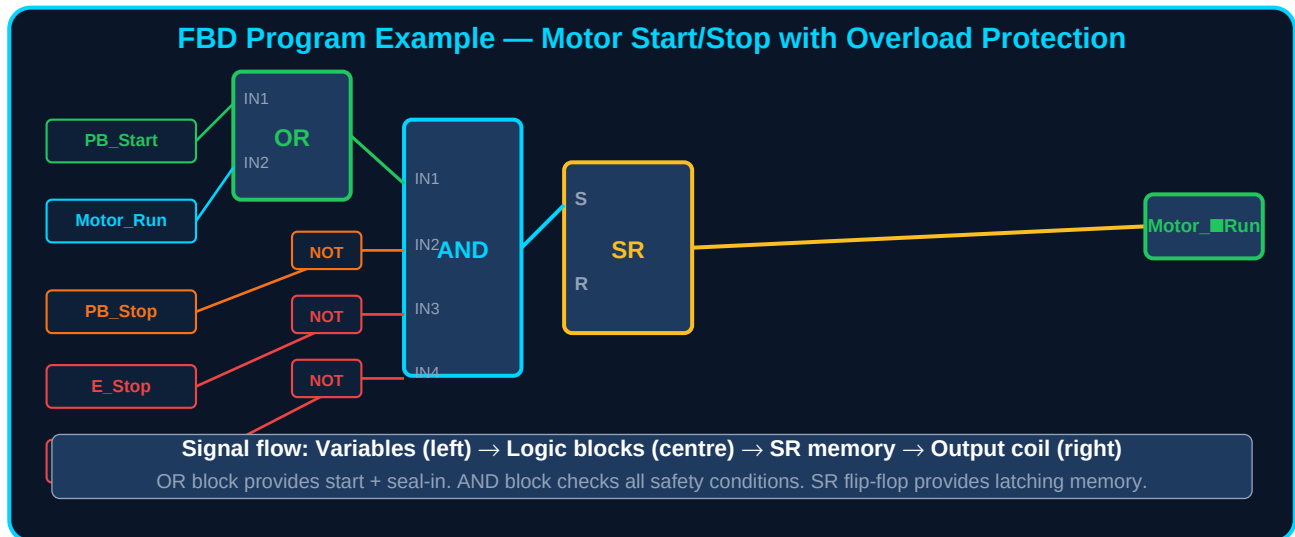
FBD Signal Types and Variable Connections

Element	Description	Example in Siemens TIA / Allen-Bradley
BOOL input wire	TRUE/FALSE signal connecting to block input. Shown as thin line.	PB_Start, Overload_OK, Sensor_Active

BOOL output wire	TRUE/FALSE output from block. Arrows show direction.	Motor_Run, Pump_Enable, Alarm_Active
INT/REAL wire	Numeric signal. Thicker wire. Used with math blocks, comparators.	Speed_Setpoint, Temperature_PV, Counter_Value
Variable at wire start	PLC tag (symbolic name) at the left end of a wire — the source of the signal.	PB_Start: BOOL — mapped to physical input I0.0
Variable at wire end	PLC tag at the right end — the destination (coil, output, memory bit).	Motor_Run: BOOL — mapped to physical output Q0.0
Block instance name	Each block placement is an instance with its own name and memory.	Timer1_TON, Counter1_CTU — unique names per instance

2. A Complete FBD Program — Motor Control

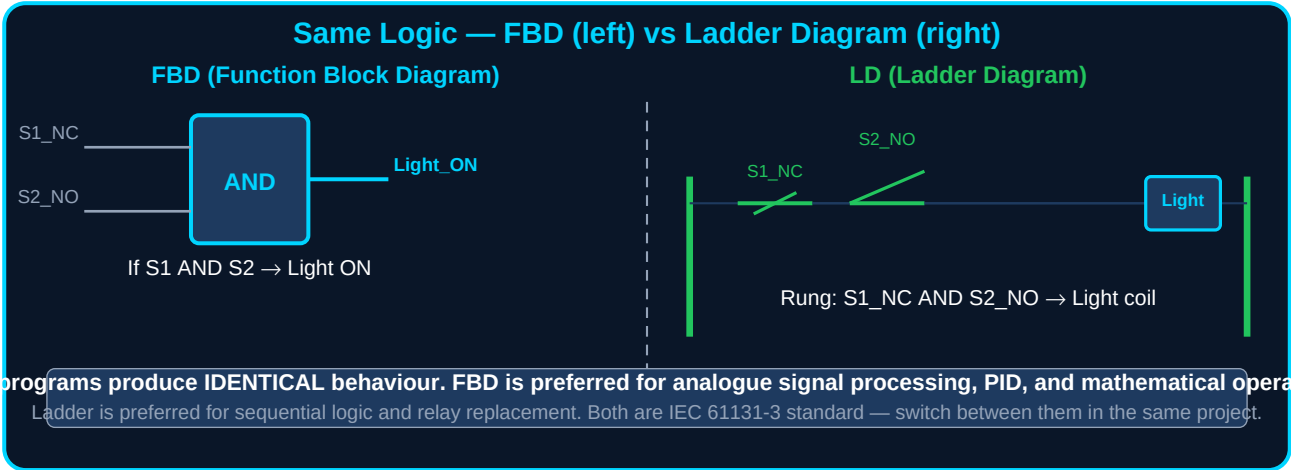
The motor start/stop circuit (with E-stop, overload, and seal-in) is the most fundamental PLC programme. In Lesson PLC-006 you saw this in Ladder Diagram. Here it is implemented in FBD — the same logic, a different visual representation.



Reading the Motor Control FBD

- **OR block (green):** Output is TRUE if PB_Start (start pushbutton pressed) OR Motor_Run (already running — seal-in contact). This creates the self-latching behaviour.
- **NOT blocks (orange/red):** Inverts the Stop pushbutton, E-Stop, and Overload signals. These devices provide Normally Closed contacts in relay logic — in FBD, they are represented as NOT blocks.
- **AND block (cyan):** All four inputs must be TRUE simultaneously: (OR output) AND (NOT Stop) AND (NOT E-Stop) AND (NOT Overload). If any condition fails, output goes FALSE.
- **SR block (yellow):** Set-Reset flip-flop. Maintains the Motor_Run state between scans. S=set (latch), R=reset (unlatch). Provides memory so the motor keeps running after the start button is released.

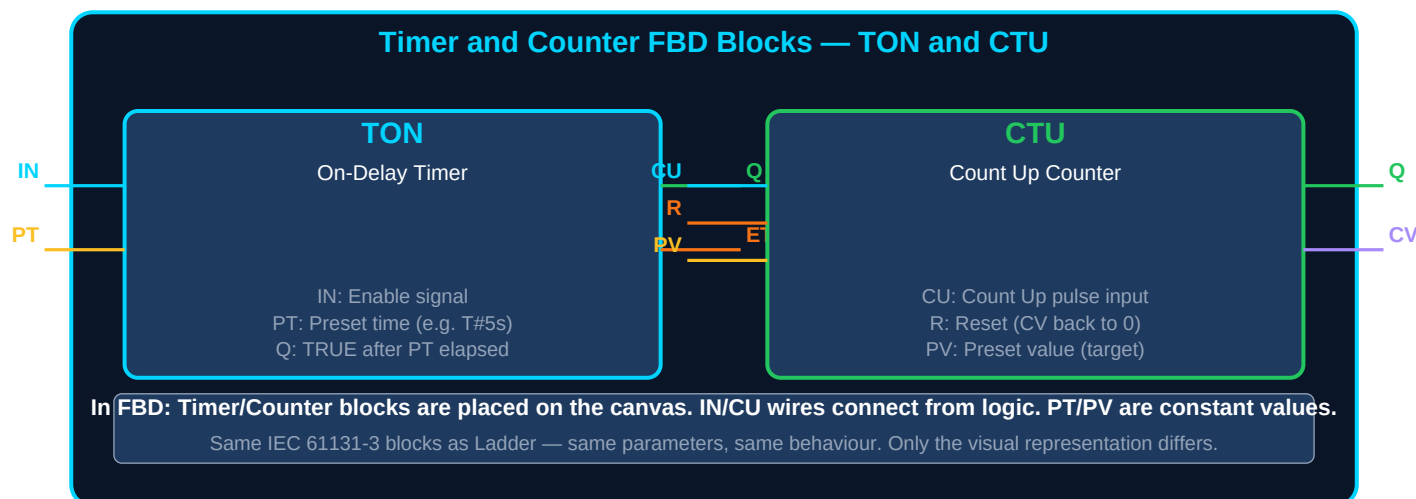
3. FBD vs Ladder Diagram — Choosing the Right Language



Criteria	Choose FBD	Choose Ladder Diagram
Logic type	Analogue signal processing, PID loops, mathematical calculations, multi-input logic networks	Discrete (ON/OFF) control, relay replacement, sequential machine logic with many rungs
Engineer background	Electronics, control systems, signal processing, instrumentation	Electrical, relay panel wiring, manufacturing automation
Visualisation preference	Data flow — see how signals transform step by step	Relay circuit — see conditions and coils in series/parallel
Debugging style	Monitor data values flowing through blocks — see intermediate results at each block output	Monitor contact states and coil energisation — see which conditions are TRUE/FALSE in real time
Typical application	Temperature control with PID, analogue I/O processing, mathematical scaling, drive speed control	Motor start/stop, conveyor logic, safety interlock sequences, batch step control

4. Timer and Counter Blocks in FBD

The IEC 61131-3 standard timer and counter function blocks are the same in all five languages — only the representation changes. In Ladder, a TON timer looks like a rectangular instruction box on a rung. In FBD, it looks like a signal-flow block with input wires for IN and PT, and output wires for Q and ET. The behaviour is identical.



Timer Block Types in FBD

Block	Full Name	Inputs	Outputs	Behaviour
TON	On-Delay Timer	IN (enable), PT (preset time)	Q (TRUE after PT), ET (elapsed time)	Q becomes TRUE PT after IN becomes TRUE. Q resets immediately when IN goes FALSE.
TOF	Off-Delay Timer	IN (enable), PT (preset time)	Q (controlled output), ET (elapsed)	Q goes FALSE PT after IN goes FALSE. Useful for motor coast-down, fan run-on.
TP	Pulse Timer	IN (trigger), PT (pulse time)	Q (pulse output), ET (elapsed)	Q goes TRUE for exactly PT when IN has a rising edge. Ignores further IN changes during pulse.
RTO	Retentive On-Delay	IN (enable), R (reset), PT (preset)	Q, ET (accumulated)	Accumulates ON time across multiple trigger events. ET only resets when R=TRUE.

Counter Block Types in FBD

Block	Full Name	Inputs	Outputs	Behaviour
-------	-----------	--------	---------	-----------

CTU	Count Up	CU (count pulse), R (reset), PV (preset value)	Q (TRUE when $CV \geq PV$), CV (current value)	Increments CV on each rising edge of CU. Q TRUE when target reached. R=TRUE resets CV to 0.
CTD	Count Down	CD (count pulse), LD (load), PV (preset)	Q (TRUE when $CV \leq 0$), CV (current value)	Decrements CV on each CD rising edge. LD=TRUE loads PV into CV. Q TRUE when CV reaches 0.
CTUD	Up/Down Counter	CU, CD, R, LD, PV	QU (at max), QD (at zero), CV	Counts up on CU rising edge, down on CD rising edge. Most flexible — used for position tracking.

FBD Timer Example — Conveyor Jam Detection

Problem: A conveyor motor runs (Motor_Run = TRUE) but a part sensor (Part_Sensor) should detect a part every 30 seconds maximum. If no part is detected for 35 seconds while the motor is running, trigger a jam alarm.

- **Logic in FBD:** Wire Motor_Run → TON block IN (start timer). Set PT = T#35s. Wire Part_Sensor → rising edge detector → TON R input (reset timer each time a part passes). Wire TON Q → Jam_Alarm output coil.
- **Result:** Every time a part passes, the sensor pulse resets the timer. If no part for 35 seconds, Q goes TRUE and triggers the alarm. When the motor stops (Motor_Run=FALSE), IN goes FALSE and the timer is held (not reset).

5. User-Defined Function Blocks (FB) in FBD

Beyond the standard IEC library blocks (AND, OR, TON, CTU etc.), you can create your own Function Blocks in FBD. A user-defined FB encapsulates a complete piece of reusable control logic — for example, a motor control FB that handles start/stop, seal-in, overload, and runtime monitoring. You then use this FB as a block in higher-level programs, just like you use the standard blocks.

Concept	Description	Practical Use
FB definition	Program Organisation Unit (POU) of type Function Block. Defined once, used many times. Has internal memory (static variables persist between calls).	Define "Motor_Control_FB" with inputs: Start, Stop, Overload, EStop. Outputs: Run, Fault, RunHours.
FB instance	Each time you place an FB block in a program, you create an instance with its own memory. Pump1_FB and Pump2_FB are two instances of the same Motor_Control_FB.	Ten pumps in a system = ten instances. Change the FB once = all ten instances update.
FB inputs (VAR_INPUT)	Variables passed INTO the block when called. Read-only inside the block.	Start: BOOL; Stop: BOOL; Overload: BOOL; E_Stop: BOOL
FB outputs (VAR_OUTPUT)	Variables passed OUT of the block. Written by the FB, read by the calling program.	Run: BOOL; Fault: BOOL; RunHours: DINT (accumulated)
FB internal variables (VAR)	Variables that exist only inside the FB, persist between calls (static memory).	Seal_In: BOOL; Fault_Timer: TON (internal timer instance)

6. Practice Exercises

#	Exercise	Instructions
1	Basic FBD logic network	In Siemens TIA Portal or PLCSIM: create an FBD Network. Place an AND block. Connect three BOOL variables to its inputs: S1_No (NO limit switch), S2_Nc (NC limit switch), PB_Enable (pushbutton). Connect the AND output to Output_Coil. Simulate and verify: all three must be TRUE.

2	FBD motor start/stop	Create the motor start/stop FBD from this lesson using your PLC software. Include: OR block (start + seal), three NOT blocks (stop, e-stop, overload), AND block, SR flip-flop. Test in simulation: verify start latches, each stop condition drops motor.
3	TON timer in FBD	Design an FBD network for: "Pump runs only when Tank_Low is TRUE AND Tank_Full is FALSE. If both conditions met for 3 seconds (to reject false triggers), start Pump_Motor." Use a TON timer with PT = T#3s. Draw and implement in your PLC software.
4	Counter FBD application	A production counter: Part_Sensor detects each product. After 100 products, trigger Batch_Done output and reset the counter. Build with CTU: CU=Part_Sensor, PV=100. Wire Q to Batch_Done and to R (auto-reset). Monitor CV value in real-time simulation.
5	FBD vs LD comparison	For the same control requirement: "If Pressure_High OR Temperature_High, activate Safety_Valve. Latch until manually reset by Reset_PB." (a) Implement in Ladder Diagram. (b) Implement same logic in FBD. (c) Compare: which is easier to debug? Which is easier to understand for a new technician?

Lesson Summary

Concept	Key Rule
FBD signal flow	Signals flow left to right. Variables at left (sources), blocks in middle, variables/outputs at right.
AND block	All inputs must be TRUE for output to be TRUE. Series logic in relay equivalent.
OR block	Any input being TRUE makes output TRUE. Parallel logic in relay equivalent.
NOT block	Inverts the signal. Used in FBD where you need to check that something is NOT happening.
SR flip-flop	Set-Reset memory. S=1 latches output TRUE. R=1 resets to FALSE. Provides motor seal-in memory in FBD.
TON in FBD	IN wire starts timer. PT = preset time constant. Q wire goes TRUE after PT. ET wire shows elapsed time.
CTU in FBD	CU wire increments on rising edge. PV = target count. Q goes TRUE when CV >= PV. R resets CV to 0.
When to use FBD vs LD	FBD: analogue processing, PID, math operations, signal flow logic. LD: discrete logic, relay replacement, sequential machine control.

What Is Next — PLC-009

PLC-009: Structured Text (ST) Programming — The most powerful IEC 61131-3 language. Structured Text is a high-level text language similar to Pascal. Covers: variable declarations, arithmetic, IF/ELSE, WHILE loops, FOR loops, CASE statements, and function calls. When ST outperforms Ladder and FBD for complex algorithms, string processing, and analogue scaling.

Resource	Details
PLC PDF Series	PLC-001 through PLC-030 — ayetechub.com/pdfs.html
Free PLC Simulator	Browser-based Ladder Logic tool — ayetechub.com/downloads
Community	Telegram: t.me/ayetechub